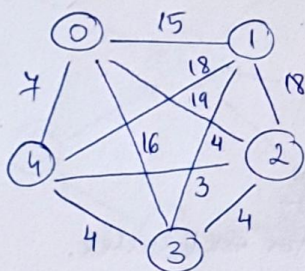


Patrîmanciu exam prep

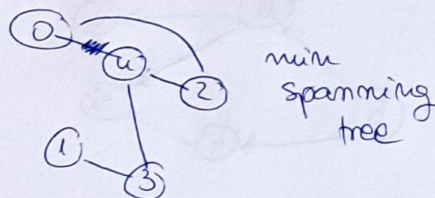
- 1) Build a path by choosing edges, drawing shows the path being built.
(Kruskal for min. spanning tree).



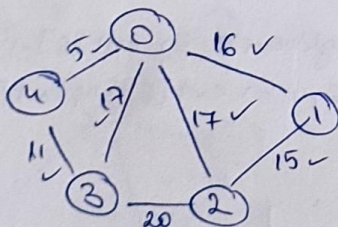
path = [0, -1, -1, -1, -1]
(Hamiltonian) [0, 2, -1, -1, -1]
↓
visit each [0, 2, 4, -1, -1]
vertex succ. ...
[0, 2, 4, 3, 1]

Sorted edges:

- ✓ (4,2) = 3 ⇒ no cycle ⇒ take it
- ✓ (1,3) = 4 ⇒ no cycle ⇒ take it
- ✓ (4,3) = 4 ⇒ no cycle ⇒ take it
- (2,3) = 4 ⇒ reject (cycle).
- (0,4) = 7 ⇒ no cycle ⇒ ~~take it~~, but deg > 2.
- (0,1) = 15 ⇒ x, deg > 2
- (3,0) = 16 → x → deg > 2
- (1,2) = 18 → x → deg > 2
- (4,1) = 19 → x → deg > 2
- ✓ (0,2) = 19 → * ✓ ⇒ no cycle, deg ≤ 2 ✓



- 2) Find Hamiltonian cycle (goes through all vertices and only repeats the first/last vertex ⇒ cycle).



Sorted edges:

- (0,4) ✓
- (4,3) ✓
- (2,1) ✓
- (0,1)
- (2,0)
- (0,3)
- (3,2) ✓

path: 0 -1 -1 -1 -1
0 -4 -3 -2 -1
0 4 -1 -1 -1
0 4 3 -1 -1
0 4 3 2 -1

0	4	3	2	1
---	---	---	---	---

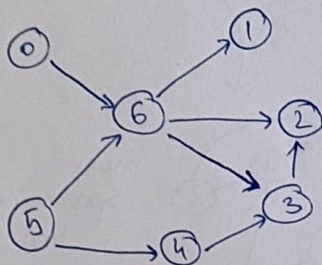
3) Topological Sorting with predecessor counting (Kahn) alg.

DAG → yes/no?

→ if yes → topological ordering?

→ if stuck alg → has cycles ⇒ not DAG.

DAG:



Predecessor counting:

(1) → compute INDEGREE for each vertex.

(2) → put all vertices with INDEGREE=0 to the queue

→ while not empty:

→ remove one vertex from queue

→ add it to the topological order

→ decrease ~~degree~~ INDEGREE of all its outbound neighbours.

(1):

vertex	0	1	2	3	4	5	6
indegree	0	1	2	2	1	0	2

(2): $Q: \{0, 5\} \rightarrow \{5\} \rightarrow \{4, 6\}$

(3)

x	q	ent
	0, 5	0 1 2 3 4 5 6
	0, 5	0 1 2 2 1 0 2
0	5	- 1 2 2 1 0 1
5	4, 6	- 1 2 2 0 - 0
4	3, 6	- 1 2 1 - - 0
6		

→ 0 has neighbour 6 ⇒ $ent[6] - 1 = 2 - 1 = 1$.

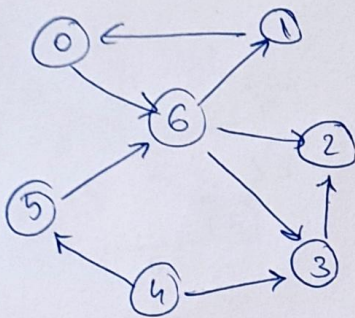
→ neighb. 6, 4 ⇒ $ent[6] - 1$; $ent[4] - 1$.

Topological order: [0]

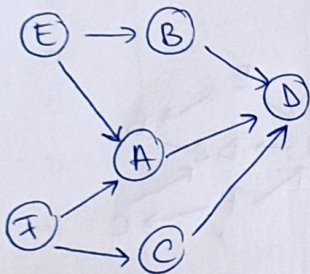
[0, 5]

... [0, 5, 4]

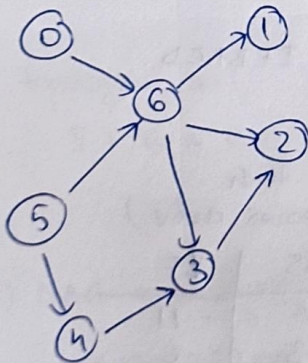
Cycle - Containing graph: (pred. count.)



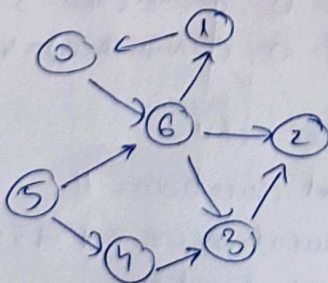
DAG: (pred. count.)



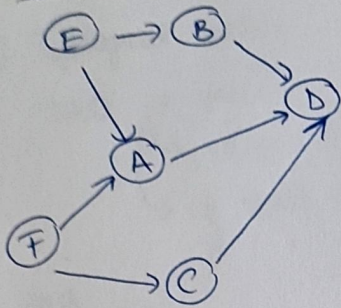
DAG: (stack)



Cycle - (stack)



MAG (stack)

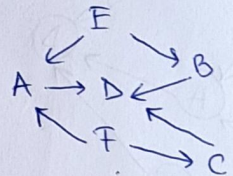
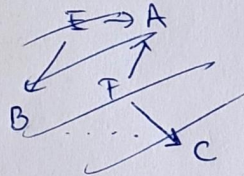


4) Scheduling.

Act.	Duration	Prereq.
A	1	E, F
B	2	E
C	3	F
D	5	A, B, C
E	4	-
F	3	-

Critical path: when $ES = LS$.

Multiple critical paths.



Topological sort deduction:

x	2	inbound
		A B C D E F
		2 1 1 3 0 0
E, A, B	1 0 1 3 - 0	
A, B, D	0 - 1 2 - 0	

x	2	inbound
		A B C D E F
		2 1 1 3 0 0
E, F		
E, F, B	1 0 1 3 - 0	
F, B, A, C	0 0 0 3 - -	
B, A, C	0 - 0 2 - -	
A, C	- - 0 1 - -	
C, D	- - - 0 - -	
D		

IFBACD.

min

(what Leaves activ.)

max

ET: (what ENTERS activ.)

Activity	Duration	ES	EF
E	4	0	4
F	3	0	3
B	2	4	4+2=6
A	1	4	4+1=5
C	3	3	3+3=6
D	5	6	6+5=11

total proj. time

Activ.	ES	EF	LS	LF
E	0	4	0	4
F	0	3	0	3
B	4	6	4	6
A	4	5	5	6
C	3	6	3	6
D	6	11	6	11

=> E, F, B, C, D critical activ.

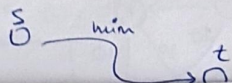
$E \rightarrow B \rightarrow D$, duration = 11

$F \rightarrow C \rightarrow D$, duration = 3+3+5=11

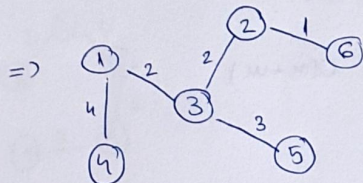
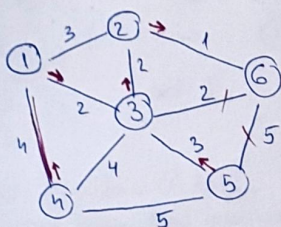
= proj. duration
=> they are both critical paths.

stra's algorithm (lowest cost walk) ✓

optimal path



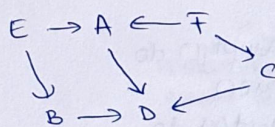
1) Min. spanning tree. → Kruskal 1) sort edges by cost.



$$\Rightarrow \text{MST Cost} = 2+2+1+3+4 = 12$$

6) ET, LT, topological sort (w proof); critical activities. (Full solution)

Activity	Duration	Prereq.
A	2	E, F
B	1	E
C	3	F
D	5	A, B, C
E	4	-
F	3	-



Topological sort deduction:

X	2	inbound					
		A	B	C	D	E	F
E, F	2	1	1	3	0	0	
E	1	0	1	3	-	0	
F	0	0	0	3	-	-	
B	0	-	0	2	-	-	
A	-	-	0	1	-	-	
C	-	-	-	0	-	-	
D							

topological sort:
E F B A C D.

7) DAG → ≠ paths of max length

if $d[y] < d[x] + 1$
 $d[y] = d[x] + 1$
 $\text{count}[y] = \text{count}[x]$
 or if $d[y] = d[x] + 1$ then
 $\text{count}[y] = \text{count}[y] + \text{count}[x]$

8) directed graph; $(s), (t) \rightarrow$ nr. of $\neq$ paths of min. length $(s) \rightarrow (t)$.
 $\Rightarrow$ BFS ; polyn. time.

Queue q

$O(n+m)$

Dictionary dist

Dictionary count

for x in X do

$dist[x] = \infty \rightarrow$ min length from (s) to (x) .

$count[x] = 0 \rightarrow$ nr. of min-length paths from (s) to (x) .

$dist[s] = 0$

$count[s] = 1$

$q.enqueue(s)$

while not $q.isEmpty()$ do

$x = q.dequeue()$

for y in $Next(x)$ do

if $dist[y] == \infty$ then

$dist[y] = dist[x] + 1$

$count[y] = count[x]$

$q.enqueue(y)$

else if $dist[y] == dist[x] + 1$ then

$count[y] = count[y] + count[x]$

return $count[t]$

9) dir. graph; costs, shortest ^{cycle} path of strictly negative total cost that starts and ends in (s) .
 If more $\Rightarrow$ take the one that minimizes the length.

10) $1 - 1$ — through given vertex.
 cycle of min length

11) $1 - 1$ — through given vertex.

9)

for x in X do:

$w[0][x] = \infty$

(*) s is unreachable.

$w[0][s] = 0$

for $k = 0, n-1$ do

for x in X do

~~$w[k+1, x] = w[k, x]$~~

$prev[k+1, x] = prev[k, x]$

for x in X do

for y in $Next(x)$ do

if $w[k+1, y] > w[k, x] + c(y, x)$ then

$w[k+1, y] = w[k, x] + c(y, x)$

$prev[k+1, y] = x$

if $w[k+1, s] < 0$ then

cycle = []

$x = s$

$i = k+1$

while $i > 0$ do

cycle.append(x)

$x = prev[i, x]$

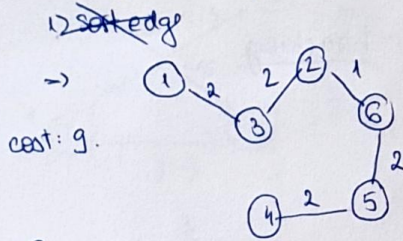
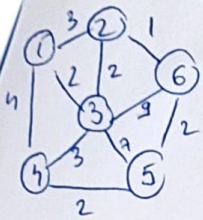
$i = i-1$

cycle.append(s)

cycle.reverse()

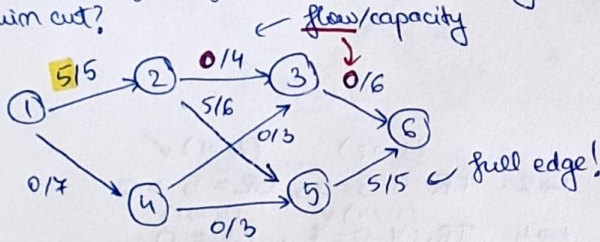
return cycle

min. spanning tree using Prim's alg.



13) max flow from ① to ⑥.

start from given flow $\rightarrow$ maximise it using Ford-Fulkerson.
min cut?

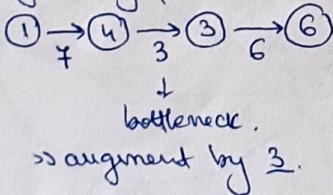


1) look at the edges with $\neq 0$ flow:
 $1 \rightarrow 2 \rightarrow 5 \rightarrow 6$ (initial flow)
current flow value: **5**.

2) residual graph: (capacities)

unused	$1 \rightarrow 4: 7$	$1 \rightarrow 2: FR = 5 - 5 = 0; BR = 5$
	$2 \rightarrow 3: 4$	$(1 \rightarrow 2) \quad (2 \rightarrow 1)$
	$2 \rightarrow 5: 1$	$2 \rightarrow 5: FR = 6 - 5 = 1; BR = 5$
	$4 \rightarrow 3: 3$	$(2 \rightarrow 5) \quad (5 \rightarrow 2)$
	$4 \rightarrow 5: 3$	$5 \rightarrow 6: FR = 5 - 5 = 0; BR = 5$
	$3 \rightarrow 6: 6$	$(5 \rightarrow 6) \quad (6 \rightarrow 5)$
	$\underline{2 \rightarrow 4}$	

3) augmenting path:

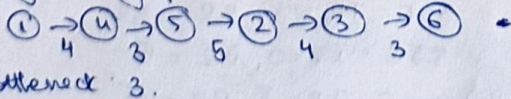


4) update original flow:

1 $\rightarrow$ 4: $0/7 \Rightarrow 3/7$; $FR = 7 - 3 = 4, BR = 3$
 $4 \rightarrow 3$: $0/3 \Rightarrow 3/3$; $FR = 3 - 3 = 0, BR = 3$
 $3 \rightarrow 6$: $0/6 \Rightarrow 3/6$; $FR = 6 - 3 = 3, BR = 3$

new flow value = initial flow + 3 = 5 + 3 = 8.

4) augmenting path:



5) orig. flow: $8 + 3 = 11$

update: $1 \rightarrow 2: 5/5$ $1 \rightarrow 4: 8/7 \Rightarrow 6/7$ $5 \rightarrow 2:$
 $4 \rightarrow 5: 3/3$

FLOW: amount sent through an edge.

CAPACITY: max allowed flow/edge.

RESIDUAL graph: modify curr. flow.

$\rightarrow$ forward edge: $cap = c - f$.

$\rightarrow$ backward edge: $cap = f$.

$3/6 \Rightarrow$ forward: $cap = 6 - 3 = 3$

$\Rightarrow$ backward: $cap = 3$.

AUGMENTING path: if every edge has positive residual cap $\Rightarrow$ more flow sent

BOTTLENECK: smallest residual cap. on the augmenting path.

aug. path: $1 \rightarrow 4 \rightarrow 3 \rightarrow 6$.

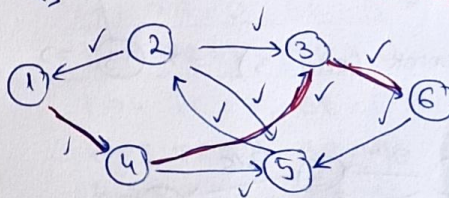
$\downarrow$
bottleneck.

CUT: partition of vertices. S/T.

source $\in S$

sink $\in T$.

sum of cap. of edges $S \rightarrow T$.

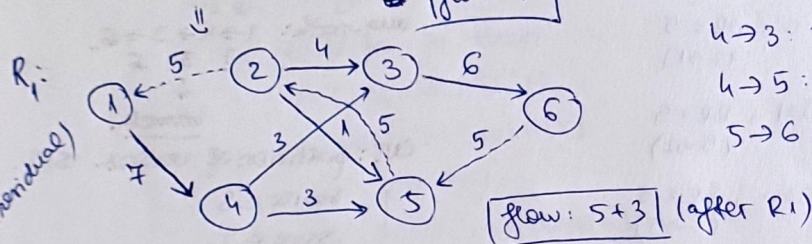
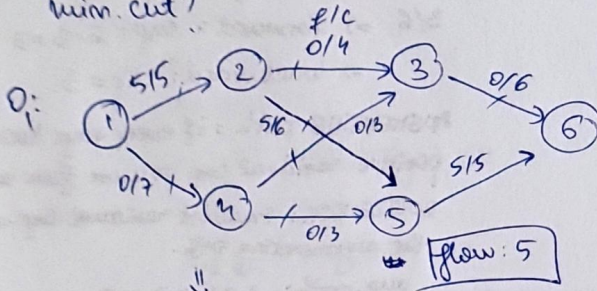


Practical work 5

Hamiltonian cycles using exact exponential bracketing

Jan 13

- 14) max. flow from $1 \rightarrow 6$.
 given flow $\rightarrow$ maximise. Ford Fulkerson?
 min. cut?

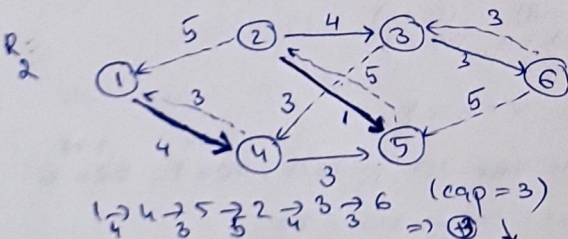
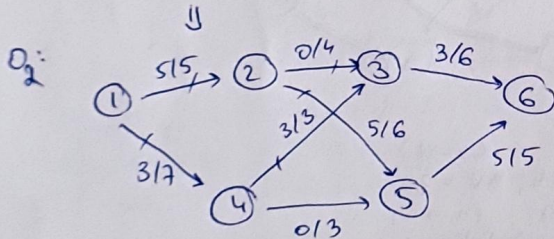


$1 \rightarrow 4 \rightarrow 3 \rightarrow 6$

bottleneck. (cap=3) $\Rightarrow$ (+3) $\Rightarrow$

(1 $\rightarrow$ 2) (2 $\rightarrow$ 1) ✓
 $1 \rightarrow 2$: FR: 5-5=0; BR=5 > 0
 $1 \rightarrow 4$: FR: 7-0=7; BR=0
 $2 \rightarrow 3$: FR: 4-0=4; BR=0
 $3 \rightarrow 6$: FR: 6; BR=0
 $2 \rightarrow 5$: FR: 1; BR=5
 $4 \rightarrow 3$: FR: 3; BR=0
 $4 \rightarrow 5$: FR=3, BR=0
 $5 \rightarrow 6$: FR=0; BR=5

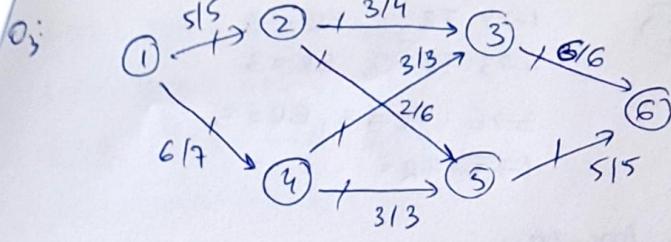
$1 \rightarrow 4$: 3/7
 $4 \rightarrow 3$: 3/3
 $3 \rightarrow 6$: 3/6



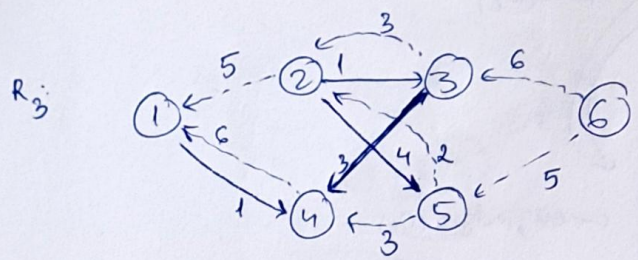
$1 \rightarrow 4 \rightarrow 5 \rightarrow 2 \rightarrow 3 \rightarrow 6$ (cap=3) $\Rightarrow$ (+3) $\Rightarrow$

✓
 $1 \rightarrow 2$: FR=0, BR=5
 $1 \rightarrow 4$: FR=7-3=4, BR=3
 $2 \rightarrow 3$: FR=4, BR=0
 $2 \rightarrow 5$: FR=1; BR=5
 $4 \rightarrow 3$: FR=0; BR=3
 $3 \rightarrow 6$: FR=3; BR=3
 $5 \rightarrow 6$: FR=0; BR=5
 $4 \rightarrow 5$: FR=3; BR=0

$\Rightarrow$ $1 \rightarrow 4$: 6/4
 $4 \rightarrow 5$: 3/3
 $5 \rightarrow 2$: 5/6 $\Rightarrow$ rev $\Rightarrow$ 5/6 - 3/6 $\Rightarrow$ 2/6. Or $2 \rightarrow 5$.
 $2 \rightarrow 3$: 4/4 $\Rightarrow$ 3/4
 $3 \rightarrow 6$: 6/6



- 1→2: FR=0, BR=5
- 1→4: FR=1, BR=6
- 2→3: FR=1, BR=3
- 2→5: FR=4, BR=2
- 4→5: FR=0, BR=3
- 4→3: FR=0, BR=3
- 3→6: FR=0, BR=6
- 5→6: FR=0, BR=5



no more paths to 6.

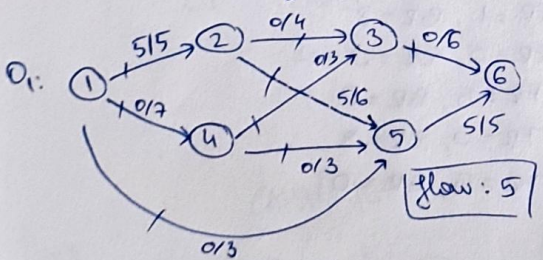
⇒ max flow: 11.

min cut: S contains source 1: $S = \{1, 4\}$
 T contains sink 6: $T = \{2, 3, 5, 6\}$

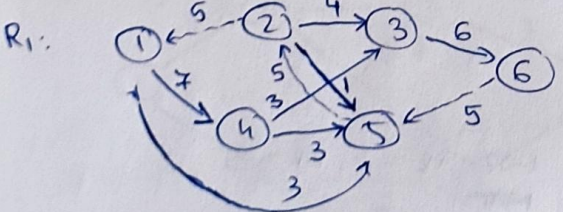
$\Rightarrow 1 \rightarrow 2 : 5$
 $\Rightarrow 4 \rightarrow 3 : 3$
 $\Rightarrow 4 \rightarrow 5 : 3$

} $\Rightarrow 5+3=11$. cut capacity.

15) max. flow ① → ⑥.
 min. cut?
 maximise using Ford-Fulkerson.

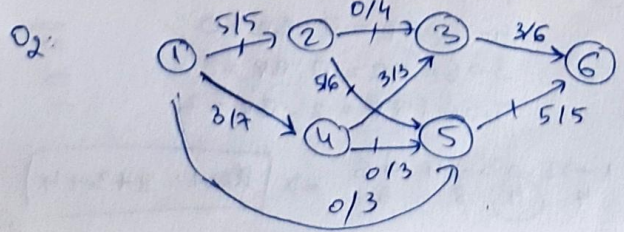


- forward residual backward residual
- 1→2: FR=0, BR=5
 - 2→3: FR=4, BR=0
 - 1→4: FR=7, BR=0
 - 4→3: FR=3, BR=0
 - 4→5: FR=3, BR=0
 - 2→5: FR=1, BR=5
 - 5→6: FR=0, BR=5
 - 3→6: FR=6, BR=0
 - 1→5: FR=3, BR=0
 - ~~0→5: FR=3, BR=0~~

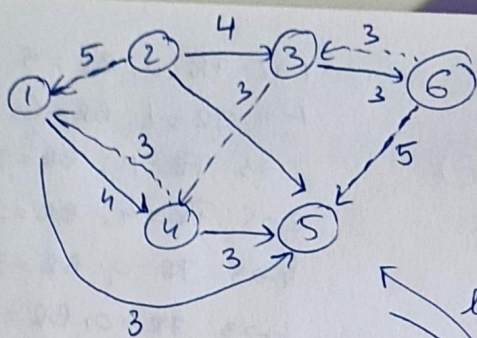


1→4→3→6 ⇒ flow: 5+3=8

$1 \rightarrow 4 : 0/7 + 3/7 = 3/7$
 $4 \rightarrow 3 : 0/3 + 3/3 = 3/3$
 $3 \rightarrow 6 : 0/6 + 3/6 = 3/6$



R₂:



$$1 \rightarrow 4: FR = 4, BR = 3$$

$$4 \rightarrow 3: FR = 0, BR = 3$$

$$3 \rightarrow 6: FR = 3, BR = 3$$

$$1 \rightarrow 5: FR =$$

look at (R₂)

done.

$$\Rightarrow \text{max. flow: } 8.$$

$$\text{min. cut: } S = \{1, 4, 5\} \quad T = \{2, 3, 6\}$$

← aug. edges (O₁).

$$1 \rightarrow 2: 5$$

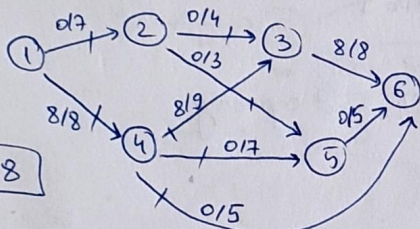
$$4 \rightarrow 3: 3$$

$$5 \rightarrow 6: 5$$

$$\Rightarrow 13 \text{ cut capacity.}$$

16)

O₁:



flow = 8

- max. flow

- maximise w Ford-Fulkerson

- min. cut.

$$1 \rightarrow 2: FR = 7, BR = 0$$

$$1 \rightarrow 4: FR = 0, BR = 8$$

$$2 \rightarrow 3: FR = 4, BR = 0$$

$$2 \rightarrow 5: FR = 3, BR = 0$$

$$4 \rightarrow 3: FR = 1, BR = 8$$

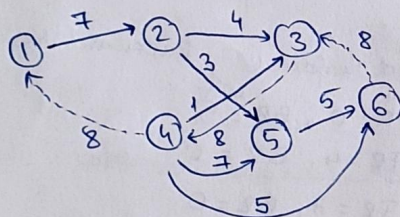
$$4 \rightarrow 5: FR = 7, BR = 0$$

$$4 \rightarrow 6: FR = 5, BR = 0$$

$$3 \rightarrow 6: FR = 0, BR = 8$$

$$5 \rightarrow 6: FR = 5, BR = 0$$

R₁:



$$1 \rightarrow 2 \rightarrow 5 \rightarrow 6$$

$$7 \quad 3 \quad 5$$

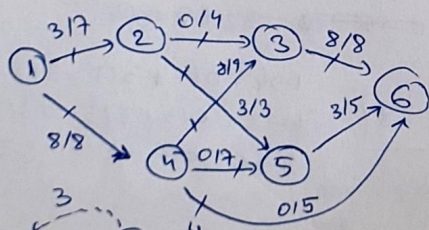
$$\hookrightarrow \text{bottleneck} \Rightarrow \text{flow} = 8 + 3$$

$$1 \rightarrow 2: 0/7 + 3/7 = 3/7$$

$$2 \rightarrow 5: 0/3 + 3/3 = 3/3$$

$$5 \rightarrow 6: 0/5 + 3/5 = 3/5$$

O₂:



$$1 \rightarrow 2: FR = 4, BR = 3$$

$$1 \rightarrow 4:$$

$$2 \rightarrow 5: FR = 0, BR = 3$$

$$4 \rightarrow 3: FR = 1, BR = 8$$

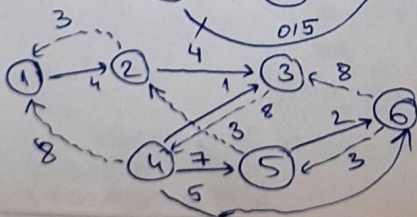
$$4 \rightarrow 5: FR = 7, BR = 0$$

$$4 \rightarrow 6: FR = 5, BR = 0$$

$$3 \rightarrow 6: FR = 0, BR = 8$$

$$5 \rightarrow 6: FR = 2, BR = 3$$

R₂:



$$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6 \Rightarrow \text{flow} = 8 + 3 + 4$$

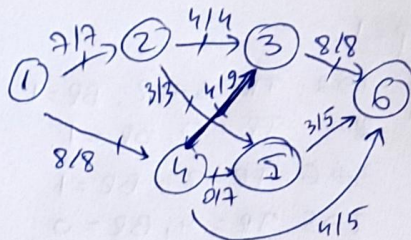
$$1 \rightarrow 2: 3/7 + 4/7 = 7/7.$$

$$2 \rightarrow 3: 0/4 + 4/4 = 4/4.$$

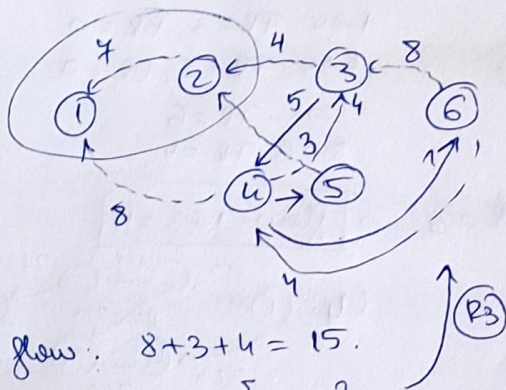
$$4 \rightarrow 5: 8/9 - 4/9 = 4/9.$$

$$4 \rightarrow 6: 0/5 + 4/5 = 4/5.$$

D_3 :



R_3 :

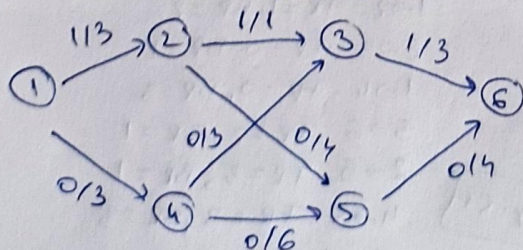
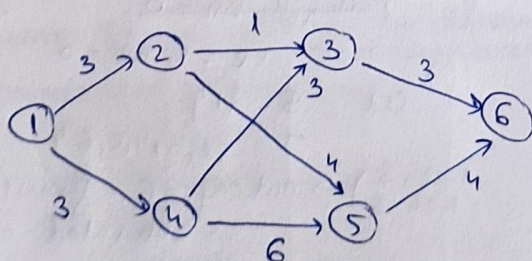


max. flow: $8+3+4=15$.

min cut: $S = \{2, 1\}$
 $T = \{3, 4, 5, 6\}$

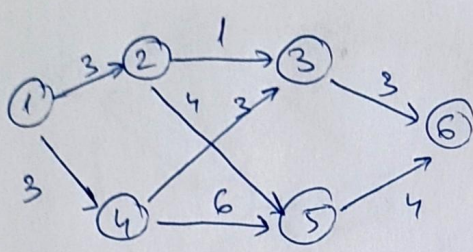
D_1 : $2 \rightarrow 3: 4$
 $2 \rightarrow 5: 3$
 $1 \rightarrow 4: 8$ } 15. cut capacity.

CAPACITIES.



$1-2-3-6$, cap:

17)

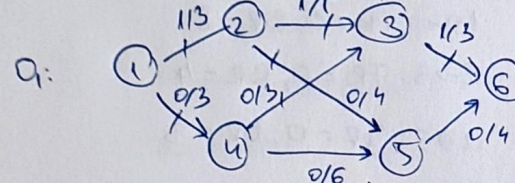


(*) max flow = 5.

min cut.

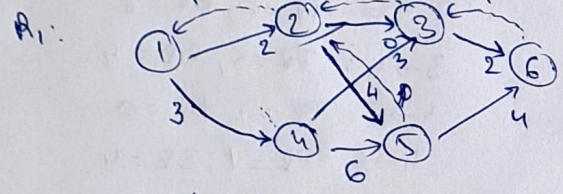
* path: (non-naive):
1→2→5→4→3→6 ⇒ ①

After augmenting path: 1, 2, 3, 6, cap = 1.



flow = 1
cap = 1

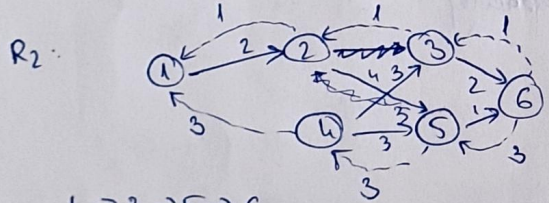
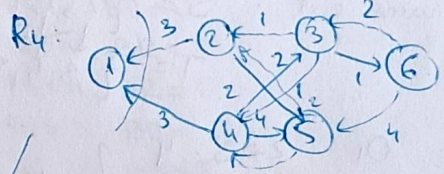
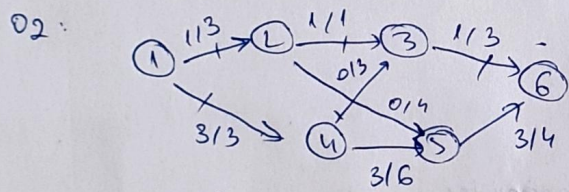
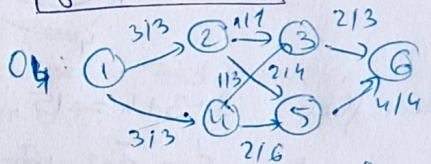
- 1→2: FR = 3-1 = 2; BR = 1
- 2→3: FR = 0, BR = 1
- 3→6: FR = 2, BR = 1
- 2→5: FR = 4, BR = 0
- 1→4: FR = 3, BR = 0
- 4→3: FR = 3, BR = 0
- 4→5: FR = 6
- 5→6: FR = 4



1→2→4→5→6: bottleneck = 3

⇒ flow: 1+3 = 4

- 1→4: 0/3 + 3/3 = 3/3
- 4→5: 0/6 + 3/6 = 3/6
- 5→6: 0/4 + 3/4 = 3/4



1→2→5→6
2 4 ① ⇒ bottleneck = 1
⇒ flow = 4+1 = 5

- 1→4: FR = 0, BR = 3
- 4→3: FR = 3, BR = 0
- 4→5: FR = 3, BR = 3
- 5→6: FR = 1, BR = 3
- 2→4: ~~FR = 0, BR = 0~~
- 2→5: FR = 4, BR = 0

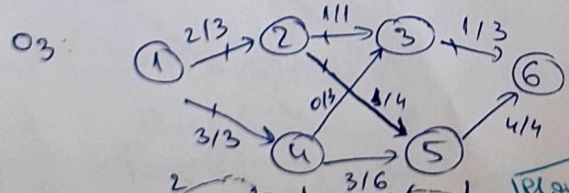
Cut: S: {1}

T: {2, 3, 4, 5, 6}

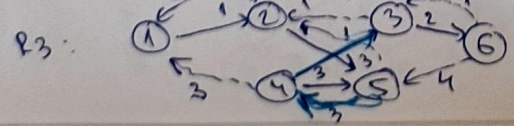
1→2: 3
1→4: 3
⇒ cut cap = 6 = flow (max)
⇒ saturated cut.

- 1→2: 1/3 + 1/3 = 2/3
- 2→5: 0/4 + 1/4 = 1/4
- 5→6: 3/4 + 1/4 = 4/4

- 1→2: FR = 1, BR = 2
- 2→3: FR = 0, BR = 1
- 1→4: FR = 0, BR = 3
- 3→6: FR = 2, BR = 1
- 2→5: FR = 3, BR = 1
- 4→5: FR = 3, BR = 3
- 5→6: FR = 0, BR = 4



flow = 5



⇒ no more paths to 6